

Reliable and Adaptive Agentic RAG Systems

Organizers

The project is organized by:

Prof. em. Dr. Gerd Kortemeyer, Michigan State University; Rectorate and AI-Center, ETH Zurich

Dr. Guang Lu, Lecturer for Data Science, Lucerne University of Applied Sciences and Arts

in collaboration with:

Dr. Diego Antognini, Senior Research Scientist, Google DeepMind

as a joint task of industry and academia for the HSLU Applied Data Science and AI Master's Program in the course **Advanced Generative AI**.

Overview

This capstone builds on the work of the previous two semesters. In the first semester, students developed a multilingual Retrieval-Augmented Generation (RAG) system, including data preparation, retrieval, re-ranking, and answer evaluation. In the second semester, students extended such systems into a multi-agent architecture in which different agents coordinated retrieval, fusion, critique, and answer synthesis.

This semester, the focus shifts from adding more techniques to studying a deeper question: **how can an agentic RAG system become more reliable and adaptive?**

A strong agentic RAG system should not only retrieve and generate answers, but also determine whether its evidence is sufficient, whether the answer is properly supported, whether the retrieved information is conflicting, whether clarification is needed, and whether it should abstain instead of producing an unreliable answer. In addition, such a system should be able to **adapt its orchestration behavior** depending on the query, the quality of the evidence, and the outcome of earlier steps.

Your task in this capstone is therefore to design, implement, and evaluate a **Reliable and Adaptive Agentic RAG system** that combines multi-agent orchestration with mechanisms such as self-reflection, trust estimation, verification, recovery, and adaptation.

Objective

Students will build on an existing strong RAG or multi-agent RAG baseline and extend it into a system that can reason not only about **what to retrieve and answer**, but also about **whether the answer should be trusted and how the system should react when uncertainty or failure is detected**.

The project is structured into five major parts:

- **Baseline Setup and Failure Analysis (15 points)** – reproducing the baseline and identifying where and why it fails
- **Reliability-Aware Agentic System Design (20 points)** – designing a multi-agent architecture that explicitly reasons about evidence quality, trust, and uncertainty
- **Reliability Mechanisms and Adaptive Recovery (25 points)** – implementing mechanisms such as verification, self-reflection, abstention, clarification, and adaptive recovery
- **Evaluation and Analysis (10 points)** – assessing system behavior quantitatively and qualitatively, including reliability and robustness
- **Final Report and Communication (5 points)** – presenting the work clearly and professionally

In addition, ambitious teams may complete optional **Extra Challenges (up to 5 bonus points)**.

The maximum total is therefore **75 points**.

The goal of the project is not only to improve answer quality, but also to study how an agentic RAG system can make **better decisions under uncertainty**.

Deadlines and Deliverables

Students form groups of size three and submit their solutions by **14th June, 2026** at the latest, passing on the codebase and the report to the [submission link](#) provided by **Dr. Diego Antognini**.

The contribution of each individual group member must be clearly stated in the submission. Students must submit:

- **Codebase / Notebook:** a well-documented implementation of the reliable and adaptive agentic RAG system
- **Final Report:** a concise report describing the design, experiments, and findings
- **Evaluation Results:** quantitative and qualitative results of the proposed system
- **Benchmark Extension:** additional benchmark cases that specifically test reliability and adaptation
- **System Demonstration:** a runnable demonstration or clearly reproducible examples showing the system's behavior

Guidance and Requirements

Step 1. Baseline Setup and Failure Analysis (15 Points)

Objective

Students will begin with a strong baseline from the previous semesters and establish a clear understanding of its behavior before extending it. The purpose of this step is to ensure that the new contributions are based on a reproducible system and motivated by concrete failure cases.

Tasks and Requirements

- **Reproduce the Baseline System**
 - Start from a strong RAG or multi-agent RAG solution from the previous semester(s)
 - Ensure that the baseline pipeline runs correctly end to end
 - Verify that the main components such as retrieval, orchestration, re-ranking, and answer synthesis are functioning properly
- **Measure Baseline Performance**
 - Report the baseline system's performance on the available benchmark
 - Include retrieval quality, answer quality, and system efficiency indicators such as latency or approximate cost
 - Clearly state which configuration is used as the reference baseline for the rest of the project
- **Analyze Failure Cases**
 - Examine representative examples where the baseline performs poorly
 - Organize these into a structured failure taxonomy, for example:
 - retrieval failure: relevant evidence was not retrieved
 - ranking failure: relevant evidence was retrieved but ranked too low
 - synthesis failure: evidence was present but the answer was still wrong or incomplete
 - grounding failure: the answer contained unsupported claims
 - ambiguity failure: the query was under-specified but the system answered anyway
 - contradiction failure: retrieved evidence conflicted and the system handled it poorly
 - orchestration failure: the system chose the wrong agent path or strategy
 - overconfidence failure: the system produced a confident answer despite weak evidence
- **Motivate the Proposed Extensions**
 - Based on the failure analysis, identify which reliability and adaptation mechanisms are most relevant for your project
 - Use these findings to justify your design choices in the next step

Deliverables

- Verified working baseline system
- Short baseline report with key metrics
- Structured failure taxonomy with representative examples
- Brief explanation of which weaknesses your proposed system aims to address

Step 2. Reliability-Aware Agentic System Design (20 Points)

Objective

Students will redesign the baseline into a reliability-aware multi-agent architecture. The aim is to move beyond simple orchestration and introduce explicit reasoning about trust, evidence quality, and uncertainty.

Tasks and Requirements

- **Design a Reliability-Aware Multi-Agent Architecture**
 - Define a set of specialized agents and their responsibilities
 - At minimum, the system should include components for:
 - query understanding or profiling
 - retrieval coordination
 - evidence analysis
 - answer generation
 - verification or critique
 - orchestration or policy control
 - The exact names of the agents may differ, but the architecture must clearly separate answer production from answer checking and decision control
- **Define Reliability Signals**
 - Specify the signals the system will use to decide whether it can proceed confidently
 - Examples include:
 - evidence sufficiency
 - agreement across retrieved chunks
 - grounding strength
 - contradiction indicators
 - ambiguity indicators
 - confidence or uncertainty scores
- **Specify the Decision Logic**
 - The system must do more than run agents in a fixed sequence
 - It should explicitly decide what to do next based on intermediate results
 - For example, the system may:
 - proceed to answer generation
 - perform another retrieval pass
 - rewrite the query
 - trigger a critique loop
 - ask for clarification
 - abstain from answering
 - Students must describe this decision policy clearly and concretely
- **Make the Orchestration Interpretable**
 - The system should record or expose why important decisions were made
 - For example, why a second retrieval pass was triggered or why the system decided to abstain

Deliverables

- Reliability-aware multi-agent RAG architecture
- Description of the agents, interactions, and orchestration logic
- Explanation of the reliability signals used by the system
- Clear justification of the design choices

Step 3. Reliability Mechanisms and Adaptive Recovery (25 Points)

Objective

Students will implement mechanisms that enable the system to behave more reliably when evidence is weak, incomplete, ambiguous, or contradictory. This is the central technical part of the project.

Tasks and Requirements

Students must implement **at least four substantial mechanisms** from the categories below. At least **one mechanism must explicitly support adaptation or recovery**, meaning that it changes the system's behavior dynamically when a problem is detected.

A. Evidence Sufficiency Estimation

Implement a mechanism to determine whether the retrieved evidence is sufficient to answer the query reliably. This may be based on semantic coverage, support from multiple chunks, missing information detection, or related criteria.

B. Groundedness / Support Verification

Implement a mechanism to check whether the generated answer is supported by the retrieved evidence. This may include claim-by-claim verification, answer-to-source alignment, citation checking, or verifier-style prompting.

C. Contradiction Detection

Implement a mechanism that detects whether the retrieved evidence contains conflicting information. The system should respond meaningfully to such cases, for example by reporting uncertainty, summarizing the conflict, retrieving additional evidence, or abstaining.

D. Clarification Strategy

Implement a mechanism that identifies under-specified or ambiguous questions and asks a clarification question instead of answering too early.

E. Abstention Mechanism

Implement a mechanism that allows the system to abstain when the available evidence is insufficient or unreliable. Abstention should be linked to explicit signals rather than used arbitrarily.

F. Self-Reflection / Critique Loop

Implement a critic, verifier, or reflective agent that reviews a draft answer and decides whether revision or another action is needed.

G. Recovery Mechanism

When a likely failure is detected, the system should be able to recover through one or more adaptive actions, for example:

- retrieving again with a rewritten query
- switching retrieval strategy
- re-ranking again
- calling a stronger verifier
- switching from answer mode to clarification mode
- switching from answer mode to abstention mode

H. Trust / Confidence Scoring

Implement a confidence or trust score for the system's answers. If such a score is used, students must explain what it means and how it is computed.

Adaptation Requirement

At least one implemented mechanism must go beyond analysis and actually **change the orchestration behavior of the system**. For example:

- route differently depending on the query type
- choose a different recovery action depending on the detected failure
- change behavior based on previous outcomes
- use feedback or past logs to improve strategy selection

Deliverables

- Integrated reliable and adaptive agentic RAG system
- Implementation of at least four reliability or adaptation mechanisms
- Clear explanation of how the system detects and responds to likely failure
- Evidence that the system adapts its behavior in meaningful cases

Step 4. Evaluation and Analysis (10 Points)

Objective

Students will evaluate the proposed system not only in terms of answer quality, but also in terms of reliability, groundedness, abstention quality, recovery behavior, and robustness on difficult queries.

Tasks and Requirements

- **Evaluate the Best System Configuration**
 - Apply the best reliable and adaptive pipeline to the benchmark
 - Compare it directly against the baseline from Step 1

- **Evaluate Retrieval and Answer Quality**
 - Use appropriate standard metrics where relevant, such as:
 - Precision@k
 - Recall@k
 - MRR
 - semantic similarity-based answer measures
 - BLEU, ROUGE, or related text comparison metrics where useful
 - Clearly explain which metrics you use and why they are appropriate
- **Evaluate Reliability-Oriented Behavior**
 - Introduce and report metrics that reflect system reliability, for example:
 - grounded answer rate
 - unsupported claim rate
 - contradiction handling quality
 - correct abstention rate
 - false abstention rate
 - recovery success rate
 - usefulness of clarification
 - alignment between confidence and correctness
 - Students may define additional well-justified metrics if they are clearly explained
- **Extend the Benchmark**
 - Add benchmark cases that specifically test reliable and adaptive behavior
 - These should include challenging cases such as:
 - ambiguous queries
 - insufficient-evidence queries
 - conflicting-evidence queries
 - adversarial or misleading queries
 - multilingual or code-switched queries, if relevant to your system
- **Perform Comparative and Qualitative Analysis**
 - Compare the proposed system to the baseline
 - Analyze whether the added mechanisms improve trustworthiness and robustness
 - Discuss tradeoffs between quality, reliability, latency, and complexity
 - Include at least one ablation study to show the contribution of an important component
 - Present representative qualitative examples, such as:
 - a successful grounded answer
 - a revised answer after critique
 - a clarification case
 - an abstention case
 - a difficult failure case

Deliverables

- Quantitative evaluation results with tables or visualizations
- Reliability-focused benchmark extension
- Comparative analysis against the baseline
- At least one ablation study
- Qualitative discussion of system behavior and limitations

Step 5. Final Report and Communication (5 Points)

Objective

Students will present their work in a clear, professional, and research-oriented manner. The report should not only describe the system, but also reflect critically on what was learned about reliable and adaptive agentic RAG.

Tasks and Requirements

- **Write a Clear Final Report**
 - Describe:
 - the baseline system

- the failure analysis
 - the proposed architecture
 - the reliability mechanisms
 - the adaptation logic
 - the experiments and results
 - the limitations and future work
- The report should be concise, structured, and analytical
- **Provide Critical Reflection**
 - Discuss which mechanisms were most useful
 - Reflect on whether self-reflection, abstention, or adaptation truly improved the system
 - Explain what did not work as expected and why
- **Ensure Professionalism and Reproducibility**
 - Code should be readable and documented
 - Experimental settings should be clear
 - Important system decisions should be inspectable through logs, outputs, or saved examples where possible

Deliverables

- Final report
- Clean and documented codebase or notebook
- Clear presentation of findings and conclusions

Suggested Benchmark Extension

To evaluate reliability and adaptation, students are encouraged to extend the benchmark beyond ordinary factual question answering. Suitable categories include:

- **Standard answerable questions**
- **Ambiguous questions** that should ideally trigger clarification
- **Insufficient-evidence questions** that should ideally lead to abstention
- **Conflicting-evidence questions** where retrieved material disagrees
- **Adversarial or misleading questions** that test robustness
- **Multilingual or code-switched questions** if appropriate for the system and corpus

The benchmark extension should be documented clearly and used in the evaluation.

Evaluation Focus

A strong project will not only produce accurate answers when evidence is strong, but also show that the system can:

- recognize when evidence is weak or incomplete
- avoid unsupported or overconfident answers
- recover intelligently when likely failure is detected
- adapt its behavior to the situation
- make its orchestration decisions more transparent

Extra Challenges (Optional, up to 5 Bonus Points)

Ambitious teams may extend their work with one or more of the following:

1. **Learning-Based Adaptive Orchestration**
Train a controller, bandit, or lightweight policy learner that improves orchestration choices based on observed success and failure.
2. **Confidence Calibration**
Study whether the system's confidence estimates align with actual correctness.
3. **Human-in-the-Loop Supervision**
Integrate or simulate human feedback to refine, correct, or override system behavior.
4. **Memory-Based Adaptation**
Allow the system to adapt decisions based on prior interactions or earlier failures.

5. **Trust-Aware Explanations**

Require the system to explain not only its answer, but also why it trusted the evidence enough to answer, revise, clarify, or abstain.

Use of Coding Tools and External Resources

Students are allowed to use coding assistants and other software-based development tools, for example for code generation, debugging, refactoring, or documentation support. However, the submission must clearly state where such tools were used and what parts reflect the students' own work.

Requirements

- **Transparency of tool usage**

Students must state which coding tools were used and for which parts of the project.

- **Clear attribution of own work**

Students must clearly distinguish between tool-supported work and their own contributions, especially regarding system design, implementation decisions, experiments, analysis, and conclusions.

- **Documentation in the report**

The final report should include a short statement describing:

- which tools were used
- how they were used
- which parts were mainly tool-assisted
- which parts were primarily developed by the students themselves

- **Citation of external resources**

If students use online resources, documentation, repositories, datasets, tutorials, or published papers, these must be cited clearly and consistently in the final report.

- **Academic integrity**

All submitted work remains the responsibility of the student team. Students must be able to explain and justify their code, design choices, experiments, and findings.